Tema 4

Utilización de objetos

Programación

Tech Hub <FP>

euroformac

ÍNDICE

Objetivos de la unidad 3
1. Introducción 4
2. Características de los objetos 5
2.1 Características de los objetos 5
3. Constructores 17
4. Instancias de objetos. Declaración y creación 20
5. Utilización de métodos. Parámetros y valores de retorno 22
5.1 Métodos de instancia 23
5.2 Métodos de clase (classmethod) 24
5.3 Métodos estáticos (staticmethod) 25
6. Utilización de propiedades 27
6.1 Propiedades de atributos de clase en Phyton: Getter, Setter y Deleter 27
7. Utilización de métodos estáticos 29
8. Almacenamiento en memoria. Tipos básicos vs objetos 30
8.1 Diferencias entre stack y heap 30
9. Destrucción de objetos y liberación de memoria 32
10. Resumen 35
Glosario 36
Bibliografía 37

Objetivos de la unidad

- Introducir los conceptos básicos de la programación orientada a objetos (POO).
- Comprender en profundidad las características de los objetos en Python, incluyendo la estructura de datos, los atributos y los métodos asociados a ellos.
- Explorar las diferentes formas de utilizar objetos en Python, desde la creación de instancias de clases existentes hasta la definición de nuevas clases con sus propias características y comportamientos.
- Analizar y aplicar los conceptos de métodos estáticos y dinámicos en Python, comprendiendo sus diferencias y usos apropiados en distintas situaciones de programación.
- Estudiar y comprender cómo se gestiona la memoria en Python, incluyendo los mecanismos de asignación y liberación de recursos, la administración de la memoria y las buenas prácticas para evitar fugas de memoria.
- Investigar y analizar la destrucción de objetos en Python, comprendiendo el proceso de liberación de memoria y la recolección de basura, y cómo optimizar el rendimiento mediante la gestión adecuada de los objetos.
- Practicar el uso de técnicas y herramientas de depuración y perfilado de código en Python para identificar y solucionar problemas relacionados con la utilización de objetos y la gestión de memoria.

1. Introducción

La programación orientada a objetos es un paradigma fundamental en el desarrollo de software. Python, como lenguaje de programación versátil y potente, ofrece herramientas y características que permiten una implementación eficiente y estructurada de este enfoque. En esta unidad, exploraremos a fondo las características de los objetos, su utilización, los métodos estáticos y dinámicos, y la utilización y liberación de memoria en Python.

Los objetos son las unidades fundamentales de la programación orientada a objetos, y en Python, se definen mediante clases. Los objetos encapsulan datos y comportamientos relacionados, y se pueden utilizar tanto objetos predefinidos en Python como la creación de nuestras propias clases para modelar entidades del mundo real o conceptos abstractos.

Los métodos estáticos y dinámicos en Python son funciones asociadas a los objetos. Son independientes de las instancias y se utilizan para operaciones que no requieren acceder a los atributos del objeto, mientras que los métodos dinámicos son específicos de cada instancia y pueden acceder a sus atributos.

La gestión de la memoria es un aspecto crítico en cualquier programa. Python ofrece un sistema de administración de memoria automático. Examinaremos cómo se asigna y libera la memoria en Python, así como las buenas prácticas para evitar fugas de memoria y optimizar el rendimiento de nuestros programas.

A lo largo de esta unidad, no solo adquiriremos conocimientos teóricos sobre las características de los objetos, su utilización, los métodos estáticos y dinámicos, y la gestión de memoria en Python, sino que también aplicaremos estos conceptos en ejercicios y proyectos prácticos

.

Tech Hub <FP> euroformac

2. Características de los objetos

2.1 Características de los objetos

Recordemos qué es un objeto y cómo podemos reconocerlo. **¿Qué es un objeto?:**

Un **objeto** en Python es una instancia específica de una clase que tiene atributos que almacenan información y métodos que definen su comportamiento y las operaciones que puede realizar. Los objetos permiten la creación de estructuras de datos y la implementación de funcionalidades específicas en programas Python.

Elementos	¿Qué Son?	¿Cómo se llaman?	¿Cómo se identifican?
Datos	Variables	Atributos	mediante sustantivos
codigo	funciones	Métodos	mediante verbos

Identificación de un Objeto

Para comenzar con las características de los objetos vamos a recordar brevemente la definición de clase debido a su íntima relación:

Definición

Las **clases** se pueden definir informalmente como plantillas (o esqueletos o planos) a partir de las cuales se crean objetos. Por ejemplo, hay millones de televisores de la misma marca y modelo en todo el mundo. Cada uno de estos televisores es idéntico en diseño/marco/modelo, por lo que los componentes, las conexiones y las características son todos idénticos. Este plano/esqueleto/modelo es una clase en la terminología de programación orientada a objetos.

Una clase en Python actúa como un plano o una plantilla que define las propiedades y los métodos que los objetos de esa clase heredarán. Define los atributos que los objetos pueden tener y los métodos que pueden ser invocados en ellos. Además, una clase puede tener también variables y métodos de clase que son compartidos por todos los objetos de esa clase.

La creación de una clase permite la creación de múltiples objetos con las mismas características y comportamientos, pero con valores de atributos

individuales. Cada objeto creado a partir de una clase se conoce como una instancia de esa clase.

La programación orientada a objetos se basa en el concepto de clases y objetos, donde las clases proporcionan la estructura y el comportamiento común, y los objetos representan instancias específicas de esa clase. Este enfoque permite la reutilización de código, la encapsulación de datos y funcionalidades, y facilita el diseño y desarrollo de programas más estructurados y modularizados.

Definición

En Python, un **objeto** es una instancia de una clase. Una clase es una plantilla o un plano que define las características y comportamientos de un tipo de objeto en particular. Un objeto se crea a partir de una clase y representa una entidad con su propio estado interno (atributos) y su conjunto de operaciones (métodos) que pueden ser realizadas sobre él.

Cada **objeto** en Python tiene un conjunto de características y comportamientos específicos, definidos por su **clase**. Las características se representan mediante atributos, que son variables asociadas al objeto y que almacenan información relevante. Los atributos pueden ser variables de cualquier tipo de datos, como enteros, cadenas de texto, listas u otros objetos.

Por otro lado, los comportamientos de un objeto se definen mediante métodos, que son funciones asociadas a la clase y que pueden ser invocadas para realizar acciones específicas sobre el objeto. Los métodos pueden acceder y manipular los atributos del objeto, así como realizar otras operaciones relacionadas con el objeto.

Un objeto en Python puede interactuar con otros objetos a través de métodos y atributos, lo que permite la creación de programas complejos y modulares. La programación orientada a objetos en Python ofrece una forma estructurada y eficiente de organizar y gestionar el código, ya que permite la reutilización de clases y la encapsulación de datos y funcionalidades.

A diferencia de los métodos procedimentales, los objetos vinculan procedimientos, así como variables y datos que se refieren a objetos. Los objetos tienen ciertas características de la vida real. Cada parte de un

programa ejecutable se trata como un objeto, lo que hace que cada parte sea independiente de las demás.

Los objetos tienen de **3 características fundamentales: métodos, eventos y atributos**.

- **Métodos:**

Los **métodos** son funciones que permiten que un objeto funcione y proporcione algún tipo de servicio durante un programa. Luego determina cómo reacciona el objeto cuando recibe un mensaje.

- **Eventos:**

Los **eventos de objetos** en Python se refieren a los eventos que ocurren en relación con los objetos y que pueden ser detectados y procesados por el código. En Python, existen varias bibliotecas y frameworks que permiten trabajar con eventos de objetos, como Tkinter, Pygame, PyQt, entre otros.

Estas bibliotecas proporcionan funcionalidades para crear interfaces gráficas interactivas y responder a eventos generados por los objetos de la interfaz. Algunos ejemplos comunes de eventos de objetos en Python incluyen clics de botones, cambios de selección en listas o menús desplegables, ingreso de texto en campos de entrada, entre otros.

Para trabajar con eventos de objetos, generalmente, se siguen los siguientes pasos:

- **Creación de objetos:** Se definen los objetos necesarios en la interfaz gráfica, como botones, campos de entrada, listas, etc.
- **Asociación de eventos:** Se asignan los eventos que se desean capturar a los objetos correspondientes. Esto se hace mediante la vinculación de funciones o métodos a los eventos específicos.
- **Definición de funciones de manejo de eventos:** Se crean funciones o métodos que se ejecutarán cuando ocurra un evento específico. Estas funciones se encargan de procesar la lógica deseada en respuesta al evento.
- **Ejecución del bucle principal:** Se inicia el bucle principal del programa, que se encarga de escuchar y procesar los eventos generados por los objetos de la interfaz.

Cuando un evento ocurre, el sistema o la biblioteca de eventos detecta el evento y lo envía a la función o método de manejo de eventos asociado. Dentro

de la función de manejo de eventos, se puede realizar cualquier acción o ejecutar cualquier código deseado en respuesta al evento.

En resumen, los eventos de objetos en Python permiten la interacción del usuario con la interfaz gráfica y el procesamiento de acciones específicas en respuesta a dichos eventos. Estas funcionalidades son proporcionadas por bibliotecas y frameworks específicos, y siguen una estructura básica de creación de objetos, asociación de eventos, definición de funciones de manejo de eventos y ejecución del bucle principal.

- **Atributos:**

Propiedades que se aplican solo a los objetos visibles para el usuario en la pantalla. Las propiedades aquí son aspectos que indican si el color, el tamaño o la posición están habilitados.

Veamos un ejemplo de objeto: un coche.

Un automóvil se considera un objeto completamente diferente al alquitrán en el suelo. Los **atributos** incluyen color, marca, modelo, número de registro y número de puertas.

Los **eventos** son todas aquellas acciones por las cuales reconocemos que le estamos dando uso al coche. Por ejemplo, abrir la puerta, girar el volante, apretar el embrague, abrir el capó.

Métodos es todo lo que nos da el coche. Señales sonoras señal al coche (evento), arranque, desaceleración al frenar.

Trabajar con objetos tiene varias ventajas:

- **Modularidad**: Al crear el código fuente para cada objeto por separado.
- **Ocultación de información**: Podemos ocultar la implementación interna interactuando solo con los métodos del objeto.
- **Reutilización**: La reutilización de objetos en Python se refiere a la práctica de utilizar objetos existentes en lugar de crear nuevos objetos desde cero. Es una técnica importante en programación orientada a objetos que permite optimizar el uso de recursos y mejorar la eficiencia del código.
- La reutilización de objetos en Python ofrece beneficios significativos, como reducir la duplicación de código, mejorar la mantenibilidad y la escalabilidad del software, y acelerar el proceso de desarrollo. Al reutilizar objetos, se fomenta la modularidad y se promueve una programación más eficiente y estructurada.

- **Facilidad de uso:** Si un determinado objeto causa problemas, simplemente programaremos la aplicación para eliminarlo o reemplazarlo.
- **Seguridad:** La posibilidad de errores disminuye, dado que no se permite la modificación directa de los datos de un objeto, sino que debemos emplear las operaciones específicas establecidas para dicho objeto

Los objetos tienen además varias características clave que los distinguen:

- **Identidad**: Característica que permite distinguir un objeto de otro. Entonces, incluso si dos objetos tienen exactamente las mismas propiedades, son diferentes. Esto podría ser una dirección de memoria, el nombre de un objeto o lo que sea que use el lenguaje para diferenciarlos. Por ejemplo, dos coches que se producen en la misma línea de producción y se ven idénticos son diferentes porque tienen un código de identificación distinto.
  En el mundo real, dos personas pueden tener los mismos rasgos como un nombre, una fecha de nacimiento o un trabajo, pero no hay duda de que son personas diferentes con sus propias identidades.
  Los objetos se pueden pensar de la misma manera. Dos objetos pueden tener el mismo estado (atributos y relaciones), pero aún existen como objetos separados con sus propias identidades.
- **Estado:** El estado de un objeto está definido por un conjunto de parámetros o propiedades que describen el objeto y sus valores. Por ejemplo, si tiene un objeto Coche, el estado es marca, modelo, color, desplazamiento, etc. Un conjunto de propiedades conocidas como atributos, junto con sus valores y cualquier conexión que el objeto tenga con otros objetos, constituyen el estado de un objeto.
  El conjunto completo de atributos del objeto y enlaces a otros objetos, no solo el atributo "estado" o un subconjunto de ellos, define el estado.
- **Comportamiento**: El comportamiento de un objeto en Python se refiere a las acciones u operaciones que puede realizar el objeto y cómo responde a ellas. En la programación orientada a objetos, el comportamiento de un objeto se define mediante los métodos asociados a su clase.
  Un método es una función que está vinculada a un objeto específico y puede ser invocada para realizar acciones o realizar operaciones sobre ese objeto. Los métodos representan el comportamiento de un objeto y pueden acceder y manipular los atributos del objeto, interactuar con otros objetos, realizar cálculos, realizar operaciones de entrada/salida,

entre otras tareas. Continuando con el ejemplo del objeto Coche, las acciones serían iniciar(), detener(), acelerar(), frenar(), etc.

Además, cómo hemos visto en la unidad 2 del curso, existen **cuatro principios básicos** de la tecnología orientada a objetos:

- Abstracción.
- Polimorfismo..
- Modularidad.
- Herencia.

Definición

La **abstracción** en Python es un concepto fundamental de la programación orientada a objetos que permite representar objetos y sus características de manera simplificada, ocultando los detalles internos y enfocándose en su comportamiento y uso.

En Python, la abstracción se logra mediante el uso de clases y objetos. Una clase es una plantilla que define las propiedades y los métodos de un objeto, mientras que un objeto es una instancia particular de esa clase.

Vemos un ejemplo sencillo que muestra cómo se puede utilizar la abstracción en Python:

```
class Vehiculo:
    def __init__(self, marca, modelo):
        self.marca = marca
        self.modelo = modelo

    def acelerar(self):
        raise NotImplementedError("Método acelerar no implementado")

    def frenar(self):
        raise NotImplementedError("Método frenar no implementado")


class Automovil(Vehiculo):
    def __init__(self, marca, modelo):
        super().__init__(marca, modelo)

    def acelerar(self):
        print(f"El automóvil {self.marca} {self.modelo} está acelerando.")
```

```
    def frenar(self):
        print(f"El automóvil {self.marca} {self.modelo} está frenando.")

automovil = Automovil("Ford", "Mustang")
automovil.acelerar()
automovil.frenar()
```

En este ejemplo, se define una clase base llamada `Vehiculo` que representa un vehículo genérico. La clase `Vehiculo` tiene dos métodos abstractos, `acelerar` y `frenar`, que se declaran sin una implementación real.

Luego, se define una subclase llamada `Automovil` que hereda de la clase `Vehiculo`. La subclase `Automovil` implementa los métodos `acelerar` y `frenar` de manera concreta, proporcionando una funcionalidad específica para un automóvil.

Al crear un objeto `automovil` de la clase `Automovil` y llamar a los métodos `acelerar` y `frenar`, se muestra la salida correspondiente.

La imagen muestra una representación visual de una clase `Vehiculo` con sus métodos abstractos `acelerar` y `frenar`, y una subclase `Automovil` que hereda de la clase base y proporciona una implementación concreta de los métodos.

Definición

El **polimorfismo** en Python es un concepto que permite que objetos de diferentes clases respondan al mismo mensaje o llamada de método de manera diferente. Es decir, el polimorfismo permite utilizar un mismo nombre de método en diferentes clases, pero con implementaciones específicas según cada clase.

Veamos un ejemplo sencillo que ilustra el polimorfismo en Python:

```
class Animal:
    def sonido(self):
        raise NotImplementedError("Método sonido no implementado")

class Perro(Animal):
    def sonido(self):
        return "Woof!"

class Gato(Animal):
```

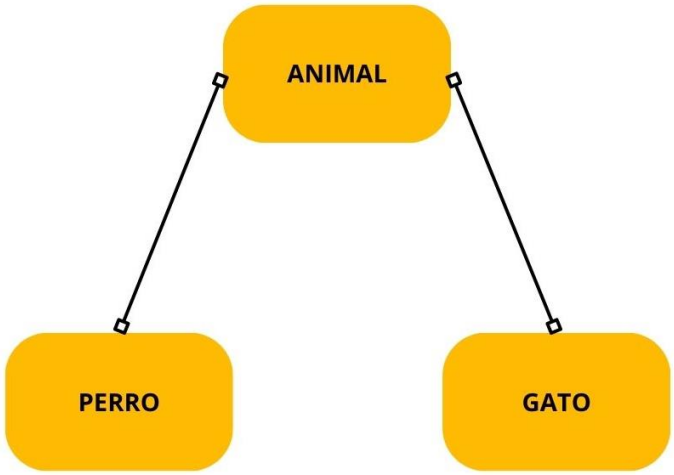
```
    def sonido(self):
        return "Meow!"
class Vaca(Animal):
    def sonido(self):
        return "Moo!"

animales = [Perro(), Gato(), Vaca()]

for animal in animales:
    print(animal.sonido())
```

En este ejemplo, se define una clase base llamada `Animal` con un método abstracto `sonido`. Luego, se definen tres subclases: `Perro`, `Gato` y `Vaca`. Cada una de estas subclases implementa el método `sonido` de manera diferente, según el sonido característico de cada animal.

A continuación, se crea una lista de objetos `animales` que contiene instancias de las tres subclases. En el bucle `for`, se llama al método `sonido` de cada objeto, y se imprime el sonido correspondiente.

El polimorfismo se refleja en este ejemplo, ya que diferentes objetos de diferentes clases (`Perro`, `Gato` y `Vaca`) responden al mismo mensaje o llamada de método (`sonido`), pero con comportamientos específicos según cada clase.

Aquí tienes una imagen conceptual que muestra el polimorfismo en Python:



Polimorfismo de las clases perro y gato

La imagen representa visualmente la idea de que diferentes objetos (`Perro` y `Gato`) pueden responder al mismo método (`sonido`) de manera diferente. Cada objeto tiene su propia implementación del método `sonido`, adaptada a las características específicas de cada clase.

Recuerda que la imagen es una representación conceptual y puede variar dependiendo del contexto y la complejidad de tu código.

Definición

La **modularidad** en Python es un principio de diseño que fomenta la subdivisión de un programa en módulos independientes y reutilizables. Se refiere a la capacidad de dividir el código en componentes más pequeños y coherentes, lo que facilita la comprensión, el mantenimiento y la reutilización del código.

En Python, los módulos son archivos que contienen código Python relacionado. Cada módulo puede tener variables, funciones, clases u otros objetos definidos en él. Los módulos se pueden importar en otros programas para utilizar su funcionalidad.

Veamos un ejemplo que ilustra la modularidad en Python:

Supongamos que tienes dos archivos: `calculadora.py` y `main.py`.

Archivo `calculadora.py`:

```
def sumar(a, b):
    return a + b

def restar(a, b):
    return a - b

def multiplicar(a, b):
    return a * b

def dividir(a, b):
    return a / b
```

Archivo `main.py`:

```
import calculadora
```

```
resultado_suma = calculadora.sumar(5, 3)
resultado_resta = calculadora.restar(10, 4)

print("Resultado suma:", resultado_suma)
print("Resultado resta:", resultado_resta)
```

En este ejemplo, el archivo `calculadora.py` actúa como un módulo que contiene funciones relacionadas con operaciones matemáticas básicas. El módulo define las funciones `sumar`, `restar`, `multiplicar` y `dividir`

En el archivo `main.py`, se importa el módulo `calculadora` utilizando la declaración `import`. Luego, se llaman a las funciones del módulo para realizar operaciones de suma y resta. Los resultados se imprimen en la consola.

La modularidad se refleja en este ejemplo, ya que se divide el código relacionado en el módulo `calculadora`, lo que permite su reutilización en otros programas. Además, al importar el módulo en `main.py`, se puede utilizar su funcionalidad de manera ordenada y clara.

Definición

La **herencia** de clases en Python es un mecanismo que permite que una clase (llamada clase hija o subclase) herede atributos y métodos de otra clase (llamada clase padre o superclase). La subclase puede agregar nuevos atributos o métodos, y también puede sobrescribir o modificar los atributos y métodos heredados de la superclase.

La herencia se logra mediante la declaración de la relación de herencia en la definición de la clase hija, utilizando la siguiente sintaxis:

```
class ClaseHija(ClasePadre):
    # Definición de la clase hija
```

Veamos un ejemplo que ilustra la herencia de clases en Python:

```
class Animal:
    def __init__(self, nombre):
        self.nombre = nombre
```

```
    def sonido(self):
        print("El animal emite un sonido genérico.")
class Perro(Animal):
    def __init__(self, nombre, raza):
        super().__init__(nombre)
        self.raza = raza
    def sonido(self):
        print("El perro ladra: ¡Guau!")
class Gato(Animal):
    def __init__(self, nombre, color):
        super().__init__(nombre)
        self.color = color
    def sonido(self):
        print("El gato maulla: ¡Miau!")
animal_generico = Animal("Animal Genérico")
animal_generico.sonido()
perro = Perro("Max", "Labrador")
perro.sonido()
gato = Gato("Luna", "Blanco")
gato.sonido()
```

En este ejemplo, se define una clase base llamada `Animal`, que tiene un método `sonido` que imprime un sonido genérico.

Luego, se definen dos subclases: `Perro` y `Gato`, que heredan de la clase `Animal`. Cada subclase tiene su propio constructor (`__init__`) y su propio método `sonido`, que sobrescribe el método heredado de la clase `Animal` con implementaciones específicas para cada animal.

Al crear instancias de las clases `Perro` y `Gato`, se puede acceder tanto a los atributos heredados de la clase `Animal` (como `nombre`) como a los atributos propios de cada subclase (como `raza` y `color`). Al llamar al método `sonido`, se utiliza la implementación correspondiente de cada subclase.

La herencia de clases en Python permite reutilizar y extender el código de manera eficiente, ya que se pueden definir características generales en la clase padre y especializarlas en las subclases.

Tech Hub <FP> euroformac

3. Constructores

Un constructor en Python es un método especial utilizado para inicializar objetos de una clase. El constructor se llama automáticamente cuando se crea un nuevo objeto y se utiliza para asignar valores iniciales a los atributos de la clase.

En Python, el constructor se define utilizando el método especial __init__()Un **constructor** en el lenguaje Python es el primer método necesario para crear una instancia de un objeto. Siempre es parte de una clase (los métodos de un objeto se definen en la clase). Los constructores siempre se llaman cuando se crea un nuevo objeto.

Los constructores de Python nos permiten establecer variables de los objetos sobre la marcha.

Veamos un ejemplo:

```
class
Mascota():
        nombre = ""
        edad = 0


    def __init__(self, nombre, edad):
        self.nombre = nombre
        self.edad = edad
        # Aquí pueden ir más cosas que se hacen en un constructor


    def saludar(self):
        print("¡Hola! Me llamo {} y mi edad es {}".format(
            self.nombre, self.edad))
    mascota = Mascota("Pepe", 3)
    mascota.saludar()
    otra_mascota = Mascota("Linda", 1)
    otra_mascota.saludar()
```

Sombreado en tono gris se indica el constructor de la clase **mascota**.

Hay algunas reglas para los constructores en Python:

- El nombre del constructor debe ser __init__: En Python, el constructor se define utilizando el método especial __init__(). Este es el nombre estándar del constructor en Python y debe ser escrito exactamente de esa manera.
- El parámetro self es obligatorio: El primer parámetro de cualquier método en una clase de Python, incluido el constructor, es el parámetro self. self es una referencia al objeto actual y se utiliza para acceder a los atributos y métodos del objeto. Debes incluir self como primer parámetro en la definición del constructor.
- El constructor no devuelve ningún valor: A diferencia de otros lenguajes de programación, como C++ o Java, en Python el constructor no devuelve ningún valor explícito. Su propósito es inicializar el objeto y no retornar datos.
- Sólo se permite un constructor por clase: En Python, una clase solo puede tener un constructor. Si defines más de un método __init__() en una clase, el último definido será el que se utilice como constructor. No se pueden definir múltiples constructores con diferentes firmas o argumentos.
- El constructor se utiliza para la inicialización de objetos: El constructor se llama automáticamente cuando se crea un nuevo objeto de la clase. Su principal función es realizar las tareas de inicialización necesarias para preparar el objeto y establecer sus atributos en un estado inicial.
- Estas reglas y convenciones son importantes para seguir las convenciones de programación de Python y asegurar que los constructores se utilicen correctamente en tus clases.

Ejemplo

La sintaxis de un constructor es def __init__(self):. Dentro de una clase se ve así:

```
class Example:
    def __init__(self):
        pass
```

Hay dos tipos de constructores en Python:

- El **Constructor predeterminado** que no tiene argumentos:

```
class myClass:
    def __init__(self):
        ...
```

- El **Constructor parametrizado que** toma uno o más argumentos:

```
class myClass:
    def __init__(self,x,y,w,h):
        ...
```

Tech Hub <FP> euroformac

4. Instancias de objetos. Declaración y creación

En la programación orientada a objetos, un objeto es una instancia de una clase. Una clase es una plantilla o estructura que define las propiedades y comportamientos que los objetos de esa clase pueden tener. Los atributos son variables asociadas a una clase u objeto que almacenan datos. Representan las características o propiedades de los objetos. Si utilizamos el ejemplo visto en la unidad 2 de la **clase Coche**, donde habíamos definido la clase como:

```
class Coche:
    def __init__(self, marca, color, matrícula,
modelo):
        # Propiedades
        self.numero_llantas = 4 # es una constante,
siempre habrá 4 llantas.
        self.marca = marca
        self.color = color
        self.matrícula = matrícula
        self.modelo = modelo
  def acelerar(self):
        print(f"Soy {self.marca}, estoy acelerando!!")
    def frenar(self):
        print(f"Soy {self.marca}, estoy frenando!!")
    def girar(self, direccion):
        print(f"Soy {self.marca}, estoy girando a la
{direccion}")
    def pitar(self):
        print('Bip Bip Bip')
```

Podemos **realizar la instanciación de los objetos** de la **clase Coche** para crear los objetos de **tipo Coche**:

```
# Objetos coche, se instancia un nivel de sangría atrás, fuera de la clase.
obj_mazda = Coche('Mazda CX5', 'Rojo', 'RED126',
'2020')
obj_renault = Coche('Renault Clio', 'Negro', 'FRE009',
'2021')
obj_audi = Coche('Audi A1', 'Blanco', 'DPK312',
'2016')
```

Y probar algunos de los atributos que hemos definido en la clase.

Objetos carro, se instancia un nivel de sangría atrás, fuera de la clase.

```
# El Mazda CX5, va a girar a la izquierda
obj_mazda.girar('izquierda')
# El Audi A1, va a girar a la derecha
obj_audi.girar('derecha')
# El Renault Clio va a acelerar
obj_renault.acelerar()
```

5. Utilización de métodos. Parámetros y valores de retorno

En la unidad 2 de este curso, hemos visto cómo crear un **método def** en una clase que acepta parámetros como entrada y cambia el estado de la instancia (como una propiedad). Con la ayuda de decoradores, puedes crear muchos modos diferentes.

Tenemos tres tipos diferentes de métodos en Python:

- **Métodos de instancia**: Son los métodos más comunes en las clases de Python. Se definen dentro de la clase y se llaman en una instancia de la clase. Estos métodos tienen acceso a los atributos y métodos de la instancia a través del parámetro `self`. No requieren ningún decorador especial para definirse, solo se definen dentro de la clase como cualquier otro método.

- **Métodos de clase:** Estos métodos están asociados con la clase en lugar de las instancias individuales. Se definen utilizando el decorador `@classmethod` y toman como primer parámetro la propia clase en lugar de la instancia (`cls` convencionalmente). Los métodos de clase pueden acceder a los atributos de clase y realizar operaciones relacionadas con la clase en lugar de las instancias individuales.

- **Métodos estáticos**: Son métodos que están definidos dentro de una clase, pero no tienen acceso a los atributos o métodos de la instancia o de la clase. Se definen utilizando el decorador `@staticmethod`. Estos métodos son independientes de cualquier instancia o clase y se comportan como funciones regulares dentro del ámbito de la clase.

En el ámbito de la programación, los métodos de instancia se emplean para llevar a cabo acciones específicas de una instancia en particular. Por otro lado, los métodos de clase son valiosos para realizar operaciones que implican a la clase en su conjunto. Por último, los métodos estáticos son útiles para ejecutar acciones que no requieren depender ni de la instancia ni de la clase en general

La siguiente sección contiene ejemplos que definen **tres tipos de métodos**:

```
class Clase:
    def metodo(self):
        return 'Método normal', self
```

```
    @classmethod
    def metododeclase(cls):
        return 'Método de clase', cls

    @staticmethod
    def metodoestatico():
        return "Método estático"
```

5.1 Métodos de instancia

Los métodos de instancia son los métodos comunes, de toda la vida, que hemos visto anteriormente. Reciben como parámetro de entrada **self** que hace referencia a la instancia que llama al método. (" ¿Qué es un método de instancia?"). También pueden recibir otros argumentos como entrada:

```
class Clase:
    def metodo(self, arg1, arg2):
        return 'Método normal', self
```

Y como ya sabemos, una vez creado un objeto pueden ser llamados:

```
mi_clase = Clase()
mi_clase.metodo("a", "b")
# ('Método normal', <__main__.Clase at 0x10b9daa90>)
```

En vista a esto, los métodos de instancia:

- Pueden acceder y modificar los atributos del objeto al que pertenecen. Utilizando la referencia "self", que representa al objeto actual, se puede acceder a los atributos y modificar su estado.
- Además, los métodos de instancia pueden acceder a otros métodos de la misma clase. Esto permite la reutilización de código y la organización de la lógica en diferentes métodos para mejorar la legibilidad y mantenibilidad del código.
- Dado que desde el objeto self se puede acceder a la clase con "self.class", los métodos de instancia también pueden modificar el estado del objeto. Esto significa que pueden actualizar los valores de los atributos del objeto durante la ejecución del método.

En resumen, los métodos de instancia tienen acceso a los atributos del objeto, pueden acceder a otros métodos de la clase y pueden modificar el estado del objeto a través de la referencia "self".

5.2 Métodos de clase (classmethod)

En Python, los métodos de clase son métodos asociados a la clase en lugar de las instancias individuales de la clase. Estos métodos se definen utilizando el decorador `@classmethod` y tienen acceso a los atributos de clase en lugar de los atributos de instancia.

La sintaxis básica para definir un método de clase en Python es la siguiente:

```
class MiClase:
    atributo_clase = "Hola"
    @classmethod
    def metodo_clase(cls, parametro1, parametro2):
        # Código del método de clase
        pass
```

En el ejemplo anterior, `metodo_clase` es un método de clase que toma dos parámetros (`parametro1` y `parametro2`). El primer parámetro, por convención, se llama `cls` y hace referencia a la propia clase. El método puede acceder a los atributos de clase utilizando `cls.nombre_atributo`.

Los métodos de clase se pueden llamar directamente en la clase sin necesidad de crear una instancia. Además, también se pueden llamar en instancias de la clase, pero en ese caso, el primer parámetro (`cls`) se pasa automáticamente y hace referencia a la clase de la instancia.

Veamos un ejemplo de cómo se utilizan los métodos de clase:

```
class MiClase:
    contador = 0

    def __init__(self):
        MiClase.contador += 1

    @classmethod
    def obtener_contador(cls):
        return cls.contador

# Crear instancias de la clase
objeto1 = MiClase()
objeto2 = MiClase()

# Llamar al método de clase para obtener el contador
print(MiClase.obtener_contador())   # Salida: 2
```

En este ejemplo, el método de clase `obtener_contador` devuelve el valor del contador de instancias. El método se llama en la clase `MiClase` y devuelve el valor 2, ya que se han creado dos instancias de la clase.

Los métodos de clase son útiles cuando necesitamos realizar operaciones que están relacionadas con la clase en sí misma y no dependen de las instancias individuales.

5.3 Métodos estáticos (staticmethod)

En Python, los métodos estáticos son métodos que están asociados con una clase, pero no tienen acceso ni a los atributos de instancia ni a los atributos de clase. Se definen utilizando el decorador `@staticmethod`.

La sintaxis básica para definir un método estático en Python es la siguiente:

```
class MiClase:
    @staticmethod
    def metodo_estatico(parametro1, parametro2):
        # Código del método estático
        pass
```

En el ejemplo anterior, `metodo_estatico` es un método estático que toma dos parámetros (`parametro1` y `parametro2`). A diferencia de los métodos de instancia y de clase, los métodos estáticos no tienen un parámetro especial para referirse a la clase o a la instancia. Puedes acceder a otros métodos o funciones dentro del método estático utilizando la notación de punto normal.

Los métodos estáticos se llaman directamente en la clase sin necesidad de crear una instancia. No tienen acceso a los atributos de instancia ni a los atributos de clase, ya que no tienen una referencia a `self` o `cls`.

Veamos un ejemplo de cómo se utilizan los métodos estáticos:

```
class Calculadora:
    @staticmethod
    def sumar(a, b):
        return a + b

    @staticmethod
    def restar(a, b):
        return a - b
```

```
# Llamar a los métodos estáticos de la clase sin necesidad
de crear una instancia
resultado_suma = Calculadora.sumar(5, 3)
resultado_resta = Calculadora.restar(10, 6)

print(resultado_suma)  # Salida: 8
print(resultado_resta)  # Salida: 4
```

En este ejemplo, la clase `Calculadora` define dos métodos estáticos: `sumar` y `restar`. Estos métodos realizan operaciones matemáticas simples y se llaman directamente en la clase `Calculadora` sin necesidad de crear una instancia.

Importante

Los métodos estáticos son útiles cuando necesitas encapsular una funcionalidad dentro de una clase, pero no depende ni de las instancias de la clase ni de los atributos de clase. Por lo tanto, puedes utilizar métodos estáticos para agrupar funciones relacionadas dentro de una clase.

6. Utilización de propiedades

6.1 Propiedades de atributos de clase en Phyton: Getter, Setter y Deleter

En Python, podemos utilizar las propiedades de atributos de clase para controlar el acceso, asignación y eliminación de los valores de los atributos de clase. Las propiedades se definen mediante el uso de decoradores, como `@property`, `@atributo.setter` y `@atributo.deleter`. Estos decoradores nos permiten definir métodos que actúan como getter, setter y deleter para los atributos de clase.

Veamos una descripción de cada uno de ellos:

- **Getter:** Se utiliza para obtener el valor de un atributo de clase. Se define utilizando el decorador `@property` sobre un método que no acepta argumentos adicionales, pero acepta `self` como primer parámetro. El método getter debe devolver el valor deseado.

```
class MiClase:
    @property
    def atributo(self):
        return self._atributo

# Uso de getter
objeto = MiClase()
valor = objeto.atributo
```

En este ejemplo, el método `atributo` se define como un getter utilizando `@property`. Cuando se accede a `objeto.atributo`, se llama al método getter y devuelve el valor del atributo `_atributo`.

- **Setter:** Se utiliza para asignar un valor a un atributo de clase. Se define utilizando el decorador `@atributo.setter` sobre un método que acepta el valor a asignar, además de `self` como primer parámetro. El método setter se llama automáticamente cuando se realiza una asignación al atributo correspondiente.

```
class MiClase:
    @property
    def atributo(self):
        return self._atributo
```

```
    @atributo.setter
    def atributo(self, valor):
        self._atributo = valor

# Uso de setter
objeto = MiClase()
objeto.atributo = 10
```

En este ejemplo, el método `atributo` se define como un getter utilizando `@property`, y el método `atributo` con el decorador `@atributo.setter` se define como el setter correspondiente. Al asignar un valor a `objeto.atributo`, se llama automáticamente al método setter y se asigna el valor al atributo `_atributo`.

- **Deleter:** Se utiliza para eliminar un atributo de clase. Se define utilizando el decorador `@atributo.deleter` sobre un método que no acepta argumentos adicionales, pero acepta `self` como primer parámetro. El método deleter se llama automáticamente cuando se utiliza la palabra clave `del` para eliminar el atributo correspondiente.

```
class MiClase:
    @property
    def atributo(self):
        return self._atributo

    @atributo.setter
    def atributo(self, valor):
        self._atributo = valor

    @atributo.deleter
    def atributo(self):
        del self._atributo

# Uso de deleter
objeto = MiClase()
objeto.atributo = 10
del objeto.atributo
```

En este ejemplo, se define el método deleter `atributo` utilizando el decorador `@atributo.deleter`. Al utilizar `del objeto.atributo`, se llama automáticamente al método deleter y se elimina el atributo `_atributo`.

Las propiedades de atributos de clase nos permiten controlar el acceso y la manipulación de los atributos, lo que brinda una mayor flexibilidad y encapsulación en nuestro código.

7. Utilización de métodos estáticos

Un método estático pertenece a una clase incluso si no depende de la clase o sus instancias. Entonces, en este caso, no necesita un parámetro predeterminado como lo haría con un método normal o de clase. Estos métodos generalmente se usan cuando están asociados con una clase de alguna manera, por lo que no es necesario mantenerlos en un módulo separado de la clase.

para declarar un método estático en Python, simplemente se utiliza el decorador @staticmethod antes de la definición del método. Esto permite que el método se llame directamente en la clase sin necesidad de crear una instancia.

```
class Math:

    @staticmethod
    def sumar(num1, num2):
        return num1 + num2

    @staticmethod
    def restar(num1, num2):
        return num1 - num2

    @staticmethod
    def multiplicar(num1, num2):
        return num1 * num2

    @staticmethod
    def dividir(num1, num2):
        return num1 / num2

print(Math.sumar(3, 4))
print(Math.restar(6, 2))
print(Math.multiplicar(5, 4))
print(Math.dividir(10, 5))
```

8. Almacenamiento en memoria. Tipos básicos vs objetos

Un objeto es una referencia o puntero a un espacio en memoria. Cuando creamos los objetos, el lenguaje orientado a objetos los guarda en la memoria y nos devuelve coordenadas con las que podremos acceder a la información que almacenamos.

Existen dos tipos de memoria: **Stack y Heap.**

- La **memoria Stack** es una opción más rápida y nos brinda la capacidad de organizar nuestra información de manera sistemática. En esta área de memoria se almacenan las variables y sus respectivos valores de tipos de datos simples, como booleanos, números y cadenas de texto, entre otros.
- Los objetos también usan memoria **Stack**, pero en lugar de almacenar información, almacenan coordenadas o referencias a la ubicación física del objeto en la pila, lo que le permite almacenar más información, pero a un ritmo más lento.

¿Qué se guarda en la memoria Stack?

- Variables locales.
- Variables de referencia.
- Parámetros y valores de retorno.
- Resultados parciales.
- El control de la invocación y retorno de métodos.

¿Qué se guarda en la memoria Heap?

- Objetos.
- Variables de instancia.

8.1 Diferencias entre stack y heap

Los segmentos que más nos interesan son HEAP y STACK, así que resumiremos las diferencias clave entre ellos en una tabla:

Parámetro	Stack (pila)	Heap (montón)
Estructura de datos	Estructura de datos lineal.	Estructura de datos jerárquica.

Velocidad de acceso	Acceso de alta velocidad.	Más lento en comparación con la pila.
Gestión del espacio	Es un espacio gestionado de manera eficiente por el sistema operativo para evitar la fragmentación de la memoria.	El espacio en la memoria Heap no se usa de manera tan eficiente. La memoria se puede dividir en bloques de memoria asignada y liberada.
Acceso	Solo variables locales.	Permite acceder a variables de forma global.
Límite de tamaño	El límite depende del sistema operativo.	No tiene un límite específico.
Redimensión	Las variables no pueden cambiar de tamaño.	Las variables pueden cambiar de tamaño.
Asignación de memoria	La memoria se asigna en un bloque contiguo.	La memoria se asigna en cualquier orden aleatorio.
Asignación y desasignación	Se realiza automáticamente mediante instrucciones del compilador.	Lo hace manualmente el programador.
Desasignación	No requiere desasignar variables.	Se necesita una desasignación explícita.
Coste	Menor.	Mayor.
Implementación	Matrices simples, memoria dinámica y basada en listas vinculadas.	Matrices y árboles.
Mayor problema	Escasez de memoria.	Fragmentación de la memoria.
Localidad de referencia	Instrucciones automáticas en tiempo de compilación.	Adecuado.
Flexibilidad	Tamaño fijo.	Redimensionable.
Tiempo de acceso	Más rápido.	Más lento.

Diferencias entre stack y heap

9. Destrucción de objetos y liberación de memoria

En Python, el proceso de destrucción de objetos y liberación de memoria es manejado automáticamente por el recolector de basura (garbage collector) incorporado en el lenguaje. El recolector de basura se encarga de identificar y eliminar los objetos que ya no están siendo utilizados en el programa, liberando así la memoria ocupada por esos objetos.

Cuando un objeto ya no tiene referencias a él (es decir, no hay variables o estructuras de datos que hagan referencia al objeto), se considera elegible para ser recolectado por el recolector de basura. El recolector de basura determina cuándo se debe realizar la recolección y libera automáticamente la memoria ocupada por los objetos inaccesibles.

El proceso de liberación de memoria no se realiza de inmediato después de que un objeto se vuelve inaccesible. En su lugar, el recolector de basura lo realiza de manera periódica, de forma transparente para el programador. La frecuencia y el comportamiento exacto del recolector de basura pueden variar según la implementación de Python utilizada.

Además, Python proporciona el método especial **__del__()** que se puede definir en una clase para realizar tareas de limpieza personalizadas antes de que el objeto sea destruido y liberado. Sin embargo, el uso de **__del__()** no es común y no se recomienda en la mayoría de los casos, ya que el recolector de basura se encarga automáticamente de la liberación de memoria.

Es importante destacar que, en Python, no tienes un control directo sobre la liberación de memoria como en lenguajes de programación de más bajo nivel como C o C++. El recolector de basura se encarga de administrar la memoria automáticamente, lo que proporciona una mayor facilidad de uso y evita problemas como fugas de memoria. Sin embargo, si estás trabajando con recursos externos, como archivos abiertos o conexiones de red, es recomendable utilizar los métodos adecuados para cerrar o liberar esos recursos de forma explícita.

Importante

En Python, el **módulo gc** proporciona funciones para controlar y trabajar con el recolector de basura (**garbage collector**). El módulo gc permite realizar acciones como habilitar o deshabilitar la recolección de basura, forzar la recolección de basura en un momento determinado y obtener información sobre los objetos y la memoria gestionados por el recolector de basura.

Algunas funciones útiles del módulo gc son:

- **gc.isenabled():** Devuelve un valor booleano que indica si la recolección de basura está habilitada o no.
- **gc.enable():** Habilita la recolección de basura.
- **gc.disable():** Deshabilita la recolección de basura.
- **gc.collect([generations]):** Inicia una recolección de basura. Si se proporciona el parámetro opcional generations, especifica el número de generaciones que se deben recolectar. Si no se especifica, se recogen todas las generaciones.
- **gc.get_count():** Devuelve una tupla que representa la cantidad de objetos gestionados por el recolector de basura en cada generación. El primer valor representa la cantidad de objetos en la generación 0, el segundo valor representa la cantidad de objetos en la generación 1 y el tercer valor representa la cantidad de objetos en la generación 2.
- **gc.get_threshold():** Devuelve una tupla que representa los umbrales de activación de recolección para cada generación. El primer valor representa el umbral de activación de recolección de la generación 0, el segundo valor representa el umbral de activación de la generación 1 y el tercer valor representa el umbral de activación de la generación 2.
- **gc.set_threshold(threshold0[, threshold1[, threshold2]]):** Establece los umbrales de activación de recolección para cada generación. Los parámetros threshold0, threshold1 y threshold2 representan los umbrales de activación para las generaciones 0, 1 y 2 respectivamente.

Estas son solo algunas de las funciones proporcionadas por el módulo gc. Ten en cuenta que, en la mayoría de los casos, no es necesario trabajar directamente con el recolector de basura y las funciones del módulo gc. El recolector de basura se encarga de administrar automáticamente la memoria de manera eficiente. Sin embargo, en casos especiales, como la optimización del rendimiento o la depuración de problemas relacionados con la memoria, las funciones del módulo gc pueden resultar útiles.

En el código lo manejaremos de la siguiente forma:

```
var = 5;
del var  # borramos a var del programa

print(gc.isenabled())  # nos dira si esta activado el
recolector de basura

gc.enable();
gc.disable()  # activa-desactiva el recolector

gc.collect()  # se forza al recolector de basura

print(gc.get_referents(objeto)  # nos da las referencias de
un objeto
```

```
gc.set_threshold(n, n, n)  # establece la frecuencia con n
print(gc.get_threshold())  # imprime la frecuencia
```

10. Resumen

La programación orientada a objetos (POO) es un paradigma de programación que se basa en la creación de objetos que tienen atributos y comportamientos asociados. En Python, un lenguaje de programación versátil y popular, la POO está ampliamente utilizada. Esta unidad de programación aborda los conceptos fundamentales de la POO en Python, incluyendo las características de los objetos, los constructores de objetos, la utilización de los métodos y la gestión de la memoria y los objetos.

Las características de los objetos y de los constructores de objetos, así como la utilización de los métodos y la gestión de la memoria y de los objetos.

En resumen, la programación orientada a objetos en Python se basa en la creación de objetos con atributos y métodos asociados. Los constructores de objetos se utilizan para inicializar los atributos de un objeto cuando se crea, y los métodos definen el comportamiento de los objetos. La gestión de la memoria y de los objetos en Python es manejada automáticamente a través de la recolección de basura.

Tech Hub <FP> euroformac

Glosario

- **Clase:** En programación, una clase es una plantilla o estructura que define un conjunto de atributos y comportamientos comunes que los objetos pertenecientes a esa clase pueden tener. Una clase sirve como un plano o molde para crear instancias de objetos.
- **Decorador**: En Python, un decorador es una construcción sintáctica que permite modificar el comportamiento de una función o clase sin necesidad de modificar su código interno. Los decoradores se utilizan para agregar funcionalidades adicionales a una función o clase de manera transparente.
- **Instancia:** En programación orientada a objetos, una instancia es un objeto específico creado a partir de una clase. Es decir, cuando se crea un objeto de una clase, se crea una instancia de esa clase. Una instancia tiene sus propios atributos y puede llamar a los métodos definidos en la clase.
- **Modularidad:** La modularidad es un principio de diseño de software que consiste en dividir un programa en módulos o unidades más pequeñas y autónomas. Cada módulo se encarga de realizar una función específica y se puede reutilizar en diferentes partes del programa, lo que facilita el mantenimiento, la legibilidad y la reutilización de código.
- **Objeto:** En programación, un objeto es una instancia de una clase. Representa una entidad o concepto con características (atributos) y acciones (métodos) asociadas. Los objetos son la base de la programación orientada a objetos y se utilizan para modelar y manipular datos de manera más eficiente y estructurada.
- **Pila:** En programación, una pila (stack) es una estructura de datos que sigue el principio de "último en entrar, primero en salir" (LIFO, Last-In-First-Out). Es similar a una pila de platos, donde solo se puede acceder al elemento superior. Los elementos se agregan y eliminan solo desde la parte superior de la pila. Se utilizan pilas en diversas aplicaciones, como la gestión de llamadas a funciones, el manejo de expresiones matemáticas y la inversión de cadenas, entre otros.

Bibliografía

"**Programación orientada a objetos**". El Libro De Python.

https://ellibrodepython.com/programacion-orientada-a-objetos

"**El tutorial de Python**". Python documentation.

https://docs.python.org/es/3/tutorial/

Tech Hub <FP> euroformac